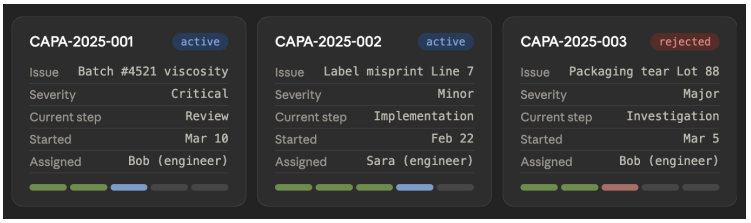
Summary:

- Key constraints to internalize and Ground work is for brainstorming and better visualisation of the assumptions and understanding along with the mind map of how i perceived the problem statement.
- The solution starts from the solution section.
- I have changed the submission format for better understanding and to add more details
- I found docs format more explanatory and flexible to add more images so used the same
- Link to git repo: https://github.com/AshBatman/unifize

Key constraints to internalize

- Every state change must be recorded with actor, timestamp, previous state, and new state
- Audit records cannot be modified or deleted, ever.
- Process definitions are configured by customers using a no-code builder
- A single customer may have dozens of active process types, each with unique steps and rules
- Multiple instances of the same process run concurrently

Ground work:

1. Assumptions:
    a. The flow is already defined by no code builder
    b. The no code builder has Process name, step definitions, field definitions, cross field rules, escalation rules and transition rules for flow defintion
    c. The process once set can be re-used for multiple instances
    i. 

    d. Every company can have a unique name and their own process that can be followed or if there are any pre-defined templates the same can be used.
    e. The no code builder output is a JSON that is sent to the Backend which is later normalised for flexibility reasons.
2. Generic User flow
    a. User builds their process using the no code builder
    b. The same is saved in the DB
    c. Once user starts the process on the UI based on rules defined the process should be followed
    d. If there are any alerts or other critical calls that are time based we will have schedulers running on a fixed time basis.
    e. We can also have triggers based on particular activities.
3. Pre-requisite

Example of no code JSON for better understanding:

```
{
  "name": "CAPA Process",
  "version": 1,
  "status": "active",

  "steps": [
    {
      "step_key": "initiation",
      "name": "1. Initiation",
      "order": 1,
      "type": "task",
      "assignee_rule": { "type": "any_user" },
      "fields": [
        { "name": "title",       "type": "text", "required": true },
        { "name": "description", "type": "text", "required": true },
        {
          "name": "severity", "type": "enum", "required": true,
          "validation": { "enum_values": ["Critical", "Major", "Minor"] }
        }
      ],
      "cross_field_rules": [],
      "escalation": null
    },
    {
      "step_key": "investigation",
      "name": "2. Investigation",
      "order": 2,
      "type": "task",
      "assignee_rule": { "type": "role", "value": "engineer" },
      "fields": [
        { "name": "findings",              "type": "text", "required": true },
        { "name": "root_cause_analysis", "type": "text", "required": false }
      ],
```

```
      "cross_field_rules": [
        {
          "if":   { "field": "severity", "op": "eq", "value": "Critical" },
          "then": { "field": "root_cause_analysis", "required": true },
          "error": "Root cause analysis is required when severity is Critical"
        }
      ],
      "escalation": {
        "timeout_hours": 48,
        "escalate_to":  { "type": "role", "value": "qa_manager" },
        "condition":    { "field": "severity", "op": "eq", "value": "Critical" }
      }
    },
    {
      "step_key": "review",
      "name": "3. Review",
      "order": 3,
      "type": "parallel_approval",
      "assignee_rule": {
        "type": "specific_users",
        "user_ids": ["u-qa-mgr-01", "u-dept-head-01"]
      },
      "fields": [],
      "cross_field_rules": [],
      "escalation": null,
      "required_approvers": ["u-qa-mgr-01", "u-dept-head-01"]
    },
    {
      "step_key": "implementation",
      "name": "4. Implementation",
      "order": 4,
      "type": "checklist",
      "assignee_rule": { "type": "role", "value": "engineer" },
      "fields": [],
      "cross_field_rules": [],
      "escalation": null,
      "checklist_items": [
        "Corrective action implemented",
        "Documentation updated",
        "Training completed",
        "Verification test passed"
      ]
    },
    {
      "step_key": "effectiveness_check",
      "name": "5. Effectiveness Check",
      "order": 5,
      "type": "approval",
      "assignee_rule": { "type": "role", "value": "qa_manager" },
      "fields": [
        { "name": "is_effective", "type": "boolean", "required": true }
      ],
      "cross_field_rules": [],
      "escalation": null
    }
  ],
```

```
  "transitions": [
    { "from": "initiation",         "to": "investigation",       "action": "complete",
"condition": null },
    { "from": "investigation",       "to": "review",              "action": "complete",
"condition": null },
    { "from": "review",              "to": "implementation",      "action": "approve",
"condition": null },
    { "from": "review",              "to": "investigation",       "action": "reject",
"condition": null },
    { "from": "implementation",      "to": "effectiveness_check",  "action": "complete",
"condition": null },
    {
      "from": "effectiveness_check", "to": "investigation",
      "action": "reopen",
      "condition": { "field": "is_effective", "op": "eq", "value": false },
      "priority": 1
    }
  ]
}
```

1. Process Definitions

ID	Name	Version	Status	Created By	Created At
d7a1f3e0-4b2c-4e8a-9f1d-a3b5c7d9e1f3	CAPA Process	1	active	u-admin-001	2025-01-15 09:00:00

2. Step Definitions

ID	Definition ID (FK)	Step Key	Name	Order	Step Type	Assignee Rule (JSON)
1	d7a1f3e0-...	initiation	1. Initiation	1	task	{"type":"any_user"}

2	d7a1f3e0-...	investigation	2. Investigation	2	task	{"type":"role","value":"engineer"}
3	d7a1f3e0-...	review	3. Review	3	parallel_approval	{"type":"specific_users","user_ids":["u-qa-mgr-01","u-dept-head-01"]}
4	d7a1f3e0-...	implementation	4. Implementation	4	checklist	{"type":"role","value":"engineer"}
5	d7a1f3e0-...	effectiveness_check	5. Effectiveness Check	5	approval	{"type":"role","value":"qa_manager"}

3. Field Definitions

ID	Step ID (FK)	Field Name	Field Type	Required	Validation (JSON)
1	1	title	text	1	null
2	1	description	text	1	null
3	1	severity	enum	1	{"enum_values":["Critical","Major","Minor"]}

4	1	findings	text	1	null
5	2	root_cause_analysis	text	0	null
6	5	is_effective	boolean	1	null

4. Cross Field Rules

ID	Step ID (FK)	Condition (JSON)	Error Message
1	2	{"if":{"field":"severity","op":"eq","value":"Critical"},"then":{"field":"root_cause_analysis","required":true}}	Root cause analysis is required when severity is Critical

5. Escalation Rules

ID	Step ID (FK)	Timeout (Hours)	Escalate To (JSON)	Condition (JSON)
1	2	48	{"type":"role","value":"qa_manager"}	{"field":"severity","op":"eq","value":"Critical"}

6. Transition Rules

ID	Definition ID (FK)	From Step	To Step	Action	Condition (JSON)	Priority
1	d7a1f3e0-...	initiation	investigation	complete	null	0
2	d7a1f3e0-...	investigation	review	complete	null	0
3	d7a1f3e0-...	review	implementation	approve	null	0
4	d7a1f3e0-...	review	investigation	reject	null	0
5	d7a1f3e0-...	implementation	effectiveness_check	complete	null	0
6	d7a1f3e0-...	effectiveness_check	investigation	reopen	{"field":"is_effective","op":"eq","value":false}	1

DB strategy for the above relations:

Table	ID Type	Rationale (Why)
process_definitions	UUID	Low volume (dozens per customer). UUID is useful if definitions are created in a

		no-code builder before hitting the DB.
step_definitions	AUTO_INCREMENT	Created server-side in batches. Integer FK JOINs to field_definitions and escalation_rules are significantly faster.
field_definitions	AUTO_INCREMENT	Pure child table, never referenced externally. Auto-increment is simpler and more performant for JOINs.
transition_rules	AUTO_INCREMENT	Child of definition; never exposed in APIs or referenced cross-service.
escalation_rules	AUTO_INCREMENT	Internal child of the step table only.
cross_field_rules	AUTO_INCREMENT	Internal child of the step table only.
process_instances	UUID (Keep)	Exposed in URLs, webhooks, and APIs. Must be globally unique and non-guessable. This is the most critical table for UUIDs.
step_instances	Either works	Referenced in step_approvals FK. Auto-increment is fine if queried via instance_id; use UUID if exposed in APIs.

step_approvals	AUTO_INCREMENT	Pure child table. Integer JOINs provide better performance.
checklist_items	AUTO_INCREMENT	Child of step_instances with no external exposure.
audit_log	AUTO_INCREMENT	500M+ rows and write-heavy. Sequential inserts are critical for InnoDB page fill and partitioning performance.

Solution:

1. Data Modeling & System Design
    a. Data policy for building this solution
        i. Transactional/Instances data vs Non transactional/Template definition data
        ii. The definitions need to be versioned and cannot be archived to maintain the history but the transactional data can have data policy for archival or broken into hot + warm DB

Aspect	Definition (Blueprint)	Instance (Runtime)
What	**Template** — defines steps, valid transitions, and field constraints.	**Live execution** — tracks current state, field values, and actor history.

Created by	Customer via no-code builder.	System (triggered when a user starts a process).
Mutability	Edited by customer (versioned/static once published).	Progresses through states, never restructured
Cardinality	One CAPA definition per customer.	Hundreds of concurrent CAPA instances.
Tables	`process_definitions`, `step_definitions`, `field_definitions`, `transition_rules`, `escalation_rules`, `cross_field_rules`	`process_instances`, `step_instances`, `step_approvals`, `checklist_items`

2. Entity Relationship - MySQL schema
    a. Total 11 tables with 3 types
        i. Defintion/Non transactional tables
        ii. Instances/ Transaction tables
        iii. Audit logs
    b. Schema diagram:
        https://dbdiagram.io/d/Unifize-69c86ad378c6c4bc7a8f1e47
3. Indexing Strategy

Index Name	Query Pattern	Rationale (Why)

idx_instances_definition	(definition_id, status)	**"Show all active CAPAs"** — This is the most common dashboard query, filtering by process type and current status.
idx_audit_instance_time	(instance_id, timestamp)	**"Full audit trail for CAPA-1234"** — This represents the hottest read path; every instance view triggers a load of this history.
idx_audit_timestamp	(timestamp)	**"All actions in March 2025"** — Essential for compliance reports that need to scan across multiple instances within a specific date range.
idx_audit_actor	(actor_id, timestamp)	**"Everything user X did last week"** — Used primarily during inspections when auditors request a history of specific actor activity.

4. State Machine Design
    a. CAPA Process Flow

    b. Transition rule examples

```
// Example 1: Simple linear -- complete initiation → investigation
{ "from": "initiation", "to": "investigation", "action": "complete", "condition":
null }

// Example 2: Rejection routing -- review reject → back to investigation
```

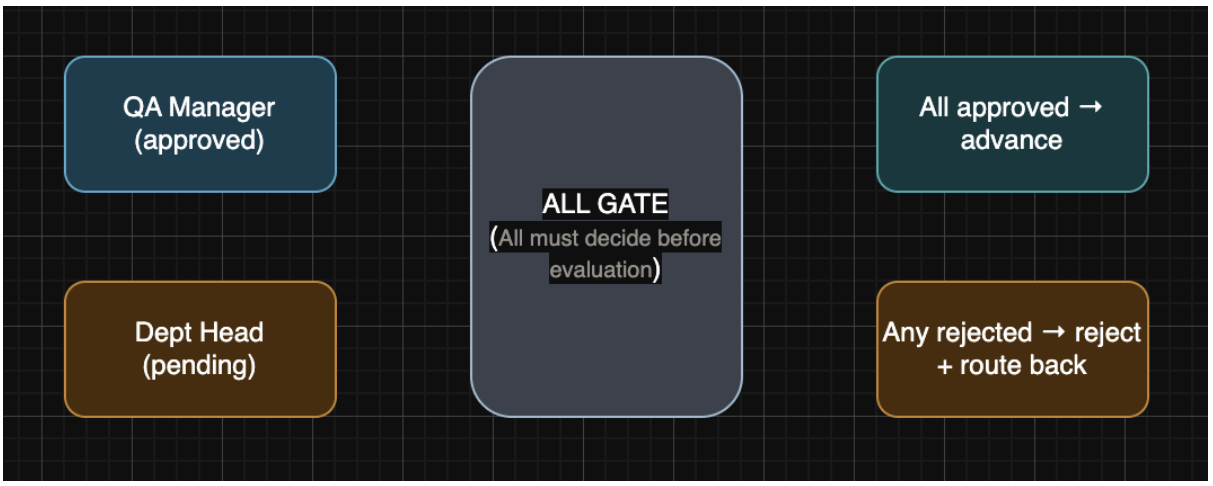
```
{ "from": "review", "to": "investigation", "action": "reject", "condition": null }

// Example 3: Conditional -- reopen only if not effective
{ "from": "effectiveness_check", "to": "investigation", "action": "reopen",
  "condition": { "field": "is_effective", "op": "eq", "value": false } }
```

c. Validation pipeline

Note: Whitelist enforcement: No matching rule = error. There is no default/fallback transition. Every valid path must be explicitly defined.

5. Parallel Approval Logic



Note: Each individual decision is a **separate audit entry**. Step stays `in_progress` until all approvers decide. One rejection immediately rejects the whole step.

6. Audit Trail Design
    a. What gets logged

Event	Logged Fields
Process started	actor, action="process_started", new_state="active", fields_snapshot
Step activated	actor, step_key, action="step_activated", new_state="in_progress"
Field updated	actor, step_key, action="field_updated", field_changes={before, after}
Step completed	actor, step_key, previous_state → new_state

Approval decision	actor, step_key, action="approved"/"rejected", comments
Escalation	actor=system, step_key, action="escalated", reason

SHA-256 Hash Chain

Each audit entry's checksum = SHA-256(previous checksum + current entry fields).

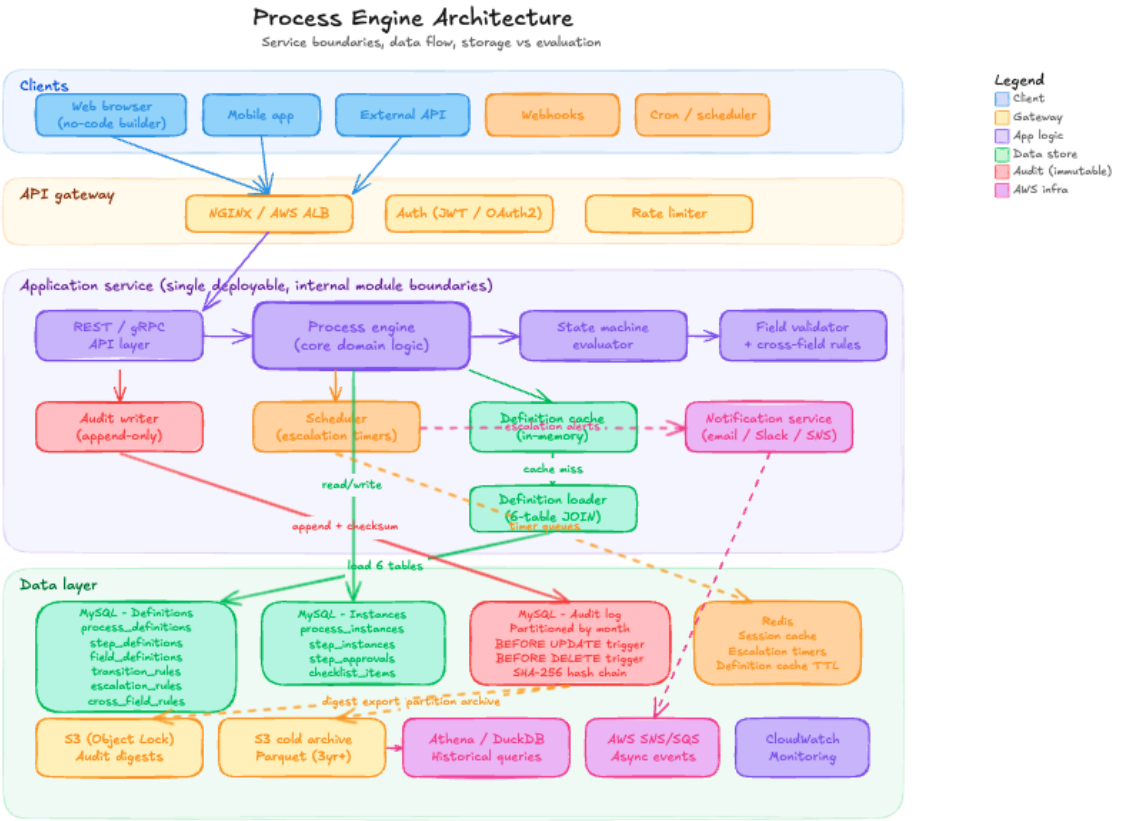
Tamper with one record → all subsequent checksums break.

b. Three Layers of Audit Immutability:

Layer	Implementation	Mechanism
Layer 1: MySQL Triggers	**Database Level**	BEFORE UPDATE and BEFORE DELETE triggers raise SIGNAL SQLSTATE '45000', physically blocking any modification at the engine level.

Layer 2: Hash Chain	**Cryptographic Level**	SHA-256 linked checksums: if any historical record is altered, all subsequent checksums become invalid, causing verification to fail.
Layer 3: Application Logic	**Code Level**	No update or delete methods exist in the service layer. The only audit operation exposed to the application is append.

7. Architecture diagram: service boundaries, data flow, definition storage vs evaluation



Implementation

1. Engine Interface
    a. Pure domain logic — no database, no HTTP server, no framework. Four methods:
2. The code is in GitHub
    a. Link:

System Evolution

Six months post-launch, three new requirements.

1. Process Versioning
    a. Update definitions without disrupting running instances.
    b. New instances get latest version.
    c. In-flight instances complete on their starting version.
    d. - **Solution**
        i. Pin definition_version on each instance at creation time. Load that exact version for all transitions. Use immutable snapshots — each version is a complete frozen copy.
    e. - Migration risks
        i. Backfill definition_version = 1 for existing instances. Step keys must be immutable across versions — new steps get new keys, removed steps marked deprecated but retained.
2. Cross-Process Dependencies
    a. Document Review can't complete until linked CAPA reaches "Implementation."
    b. Dependency Model
        i. New process_dependencies table: instance + step → depends_on_instance + step. Engine checks before advancing. When any step completes, check if it satisfies a dependency elsewhere.
    c. Circular Prevention + Cancellation
        i. DFS from target → source before creating dependency. If cycle found, reject. If blocking process is cancelled, mark dependency broken, step enters needs_review — human decides.
3. Audit Trail at 500M Rows
    a. First we will have to keep all the non transactional data in a separate DB or not consider the same for archival
    b. All the transactional data can follow Hot + Warm + Cold

Tier	Data Age	Storage Engine & Media	Latency

HOT	Last 6 months	MySQL (InnoDB), SSD, full indexes with partitioning by client IDs or by daterange and Aurora read replica with auto scaling with QEP for all select queries	< 100ms
WARM	1 year	MySQL (InnoDB), cheaper storage	< 1s
COLD	1+ years	Parquet on S3 → query via Athena Have a queueing mechanism with results stored in Redis for reading the data for later	Seconds